Smart Sleeve Hardware Setup Guide

This guide covers how to prepare the SparkFun Thing Plus ESP32 WROOM, the MyoWare 2.0 Muscle Sensor, and the AS5600 magnetic encoder to stream real data to the Smart Sleeve App over Bluetooth Low Energy (BLE).

📦 Required Components

1. **Microcontroller:** SparkFun Thing Plus - ESP32 WROOM (DEV-15663)
2. **EMG Sensor:** MyoWare 2.0 Muscle Sensor
3. **Angle Sensor:** AS5600 magnetic rotary position sensor
4. **Power/Wiring:** Jumper wires and a micro-USB cable for flashing/power.

⚡ 1. Wiring the ESP32 & MyoWare

The app is expecting the MyoWare **Envelope (ENV)** signal to be sent on **GPIO 34**. The ENV output provides a smooth, rectified curve of muscle activation, replacing the need for complex on-device DSP.

MyoWare 2.0 Pin	SparkFun ESP32 Pin	Purpose
+ (VIN)	**3V3**	Powers the sensor (Do **NOT** use 5V or USB pin)
- (GND)	**GND**	Common Ground
ENV	**34 (A2)**	Analog Output (Signal Envelope)

AS5600 Magnetic Encoder (Angle Sensor)

The knee flexion angle is measured by the **AS5600** sensor over **I2C**. The firmware reads the **RAW ANGLE** register and sends the raw 12-bit value over BLE.

AS5600 Pin	SparkFun ESP32 Pin	Purpose
VDD3V3 / VDD5V	**3V3**	Power the sensor from 3.3V
GND	**GND**	Ground
SDA	**21**	I2C data
SCL	**22**	I2C clock

AS5600 Pin	SparkFun ESP32 Pin	Purpose
DIR	**GND** or **3V3**	Fixed direction select
OUT / PWM	Not connected	Not used by this firmware
PGO	Not connected	Not used by this firmware

> [!NOTE]
> The AS5600 default I2C address is `0x36`. This firmware does not use the analog or PWM output path; it reads the digital RAW ANGLE register pair over I2C.

> [!WARNING]
> **Voltage Limits**: The ESP32's Analog-to-Digital Converter (ADC) can only handle a maximum of **3.3V**. Ensure all sensors are powered from the **3V3** pin, NOT the 5V/VUSB pin, to prevent damaging the GPIO pins.

💻 2. Preparing the Arduino IDE

We will be flashing the ESP32 using the Arduino IDE. You need the correct board manager and BLE library.

1. **Install the ESP32 Board**
   - Go to **Arduino > Settings** and add this to "Additional Boards Manager URLs": `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`
   - Go to **Tools > Board > Boards Manager**, search for `esp32`, and install it.
   - Under **Tools > Board**, select **SparkFun ESP32 Thing Plus**.
2. **Install the NimBLE Library**
   - The standard ESP32 BLE library uses too much memory. We strictly use **NimBLE**.
   - Go to **Sketch > Include Library > Manage Libraries**.
   - Search for **"NimBLE-Arduino"** (by h2zero) and install **version 1.4.x** — do not install v2.x, as the API is incompatible with this firmware.
3. **No extra AS5600 library is required**
   - The firmware talks to the AS5600 directly over Arduino `Wire`, so you do not need to install a separate AS5600 helper library.

🚀 3. Flashing the Firmware

I have provided the complete, app-compatible firmware inside the repository.

1. Open the file smart-sleeve-app/firmware/esp32_ble_sleeve.ino in your Arduino IDE.
2. Connect the ESP32 to your Mac over USB.

3. Select the correct COM/Serial Port under **Tools > Port**.
4. Click **Upload**.

This firmware has already been compile-validated against the **SparkFun ESP32 Thing Plus** target with `arduino-cli`.

> [!TIP]
> If the upload fails to connect, you may need to hold down the **"BOOT"** button on the SparkFun board when you see the `Connecting...` text in the Arduino console.

Upload Troubleshooting

If Arduino says this:

```
Failed to connect to ESP32: Wrong boot mode detected (0x13)! The chip needs to be in download mode
```

the sketch already compiled, but the ESP32 did **not** enter flashing mode.

Try this exact sequence:

1. Click **Upload** in Arduino IDE.
2. As soon as the console shows `Connecting...`, press and hold the **BOOT** button on the ESP32.
3. Keep holding **BOOT** until the upload actually starts.
4. If needed, tap **RST / EN** once while still holding **BOOT**, then release **BOOT** after the connection begins.

If it still fails:

1. Make sure **Tools > Board** is set to **SparkFun ESP32 Thing Plus**.
2. Make sure the selected serial port is the actual ESP32 device.
3. Close any open **Serial Monitor** window before uploading.
4. Try a different USB cable, because many cables provide power only and do not carry data reliably.
5. In **Tools**, reduce the upload speed if your setup is unstable.

> [!NOTE]
> The `BT: forcing BR/EDR max sync conn eff to 1...` line is only a compile-time message from the ESP32 Bluetooth stack. It is not the error causing the upload failure.

📱 Android Build Troubleshooting (Windows)

If you get an error saying **"SDK location not found"** when running `npx expo run:android`:

1. Find your Android SDK path (usually `C:\Users\<YourUser>\AppData\Local\Android\Sdk`).
2. Create a file named `local.properties` inside the `smart-sleeve-app/android/` folder.
3. Add this line to the file, using double backslashes:

```
sdk.dir=C:\\Users\\<YourUser>\\AppData\\Local\\Android\\Sdk
```

4. Restart your terminal and run the command again.

📱 4. Connecting to the App

Now that the hardware is broadcasting, let's connect it to your phone.

1. Open your project `.env` file and **turn off Mock mode**:

   ```
   EXPO_PUBLIC_USE_MOCK_HARDWARE=false
   ```

2. Because BLE requires native Bluetooth bindings, you **cannot test this inside Expo Go**. You must run a development build to your physical iPhone/Android:

   ```
   npx expo run:ios --device
   # OR
   npx expo run:android
   ```

3. Open the app and go to the BLE connection screen you use for hardware bring-up.
4. Click **Scan for Devices**. You should see `SMART-SLEEVE-01` appear.
5. Tap to connect.
6. Verify that packet counters begin increasing and the knee angle changes as you move the joint.
7. Switch to the **Dashboard** to confirm the live graphs respond to real muscle activity.

Real-hardware workflow

Use this sequence when a teammate needs to reproduce hardware bring-up without extra context:

1. Flash the ESP32 firmware and confirm the Serial Monitor shows live `[DATA]` lines.
2. Set `EXPO_PUBLIC_USE_MOCK_HARDWARE=false` in the app `.env` file.
3. Start Metro from `smart-sleeve-app/smart-sleeve-app` with:

   ```
   npx expo start --dev-client
   ```

   If the phone cannot reach Metro reliably, use:

   ```
   npx expo start --dev-client --tunnel
   ```

4. Install a native build on a physical phone:

```
npx expo run:ios --device "<IPHONE_NAME>"
# OR
npx expo run:android --device "<ANDROID_MODEL>"
```

5. Open the BLE debug screen in the app.
6. Tap **Scan for Devices**, connect to the sleeve, and watch the transport diagnostics section first.
7. Only after transport counters are healthy should you trust dashboard-level signal visuals.

Expected BLE contract

The app and firmware are currently aligned on this BLE contract:

- Device name prefix: `SMART-SLEEVE`
- Service UUID: `e0d10001-6b6e-4c52-9c3b-6a8e858c5d93`
- EMG characteristic UUID: `e0d10002-6b6e-4c52-9c3b-6a8e858c5d93`
- IMU/angle characteristic UUID: `e0d10003-6b6e-4c52-9c3b-6a8e858c5d93`
- EMG packet length: `22` bytes
- IMU packet length: `12` bytes
- Byte order: little-endian
- Checksum: XOR byte checksum
- Sample rate assumption in app: `50 Hz`
- Angle fault sentinel: `0x7FFF`

If the firmware changes any of those fields, the parser tests and the debug screen expectations need to be updated in the app as well.

What the firmware is sending

- **EMG**: one live ADC channel on CH1 from the MyoWare ENV pin, channels 2-8 set to zero
- **Angle**: raw **AS5600 12-bit RAW ANGLE** value in the IMU angle field
- **Pitch/Yaw**: always zero in this firmware
- **Fault handling**: if the AS5600 read fails or no magnet is detected, the firmware sends `0x7FFF` as the angle sentinel

Healthy connection checklist

On a healthy real connection in the BLE debug screen, you should see:

- `Requested: real`
- `Active: real`
- `Phase: connected`
- `Reconnect attempts: 0` after a steady connection is established
- `EMG packets` increasing continuously
- `IMU packets` increasing continuously

- `Last EMG age` and `Last IMU age` staying low and well under the stale timeout
- `EMG stale: no` and `IMU stale: no`
- `Characteristics` listing both BLE characteristic UUIDs above

For this firmware specifically, EMG traffic is expected on CH1 only. Channels 2-8 staying near zero is normal.

Serial Monitor sanity check

If you open the Arduino Serial Monitor at `115200`, you should see lines like:

```
[DATA] EMG=1832  AngleRaw=1024 (OK)
```

Possible angle statuses are:

- `OK` : angle read succeeded
- `MAGNET_WEAK` : angle read succeeded, but the magnet field is weak
- `MAGNET_STRONG` : angle read succeeded, but the magnet field is too strong
- `NO_MAGNET` : the AS5600 does not detect a usable magnet
- `I2C_ERROR` : ESP32 could not read the AS5600 over I2C

If the app connects but the knee angle stays at zero while the Serial Monitor shows `NO_MAGNET` or `I2C_ERROR`, the problem is on the hardware side, not in the BLE packet format.

Interpreting app-side failures

Use the BLE debug screen counters and the connector logs together:

- **Checksum mismatch**:
  - Meaning: packet length and header were valid enough to parse, but the XOR checksum failed.
  - What you will see: checksum counters increase, data may still appear, connector logs `EMG checksum mismatch` or `IMU checksum mismatch`.
  - Likely cause: corrupted bytes on the wire or firmware checksum bug.
- **Parser mismatch / invalid packet**:
  - Meaning: the packet could not be parsed at all.
  - What you will see: dropped-packet counters increase, connector logs `Dropping invalid EMG packet` or `Dropping invalid IMU packet`.
  - Likely cause: wrong packet length, wrong header, or firmware payload shape drift.
- **Transport failure**:
  - Meaning: BLE notifications or connection state failed before packet parsing could succeed.
  - What you will see: notify-error counters increase, phase changes to `reconnecting` or `failed`, connector logs notification errors or disconnect messages.
  - Likely cause: BLE instability, power issues, device reset, or native Bluetooth stack problems.
- **Discovery contract failure**:

- Meaning: the app connected to a BLE device, but it did not expose the expected sleeve service or both required characteristics.
- What you will see: phase changes to `failed` with `missing-service` or `missing-characteristics`.
- Likely cause: wrong firmware, wrong board, or a device advertising the sleeve name without the expected GATT layout.

📦 5. Payload Capture Handoff For The Electrical Team

Yes, we still need fixture captures from hardware. The app and firmware are aligned now, but fixture captures are the final bridge between simulated testing and real-world packet validation.

What we need captured

Please capture BLE notification payloads for these scenarios:

1. **Idle stream**
   - Sleeve powered, app or BLE tool connected, no muscle contraction, no deliberate knee movement.
2. **Single quadriceps contraction**
   - One clear contraction and release.
3. **Squat sequence**
   - A short continuous movement sequence with changing angle values.
4. **Encoder fault / sensor disconnected**
   - Anything that causes the AS5600 to stop producing a valid angle so the firmware sends the fault sentinel.
5. **Checksum-bad sample**
   - If they intentionally modify firmware or use a BLE injection tool to produce one bad-checksum sample, capture that too.

Preferred capture method

Preferred tool: **nRF Connect** on iPhone or Android.

Use it like this:

1. Flash the provided firmware and power the sleeve.
2. Open **nRF Connect**.
3. Scan for `SMART-SLEEVE-01`.
4. Connect and open the service `e0d10001-6b6e-4c52-9c3b-6a8e858c5d93`.
5. Enable notifications on both:
   - `e0d10002-6b6e-4c52-9c3b-6a8e858c5d93` for EMG
   - `e0d10003-6b6e-4c52-9c3b-6a8e858c5d93` for angle/IMU
6. Start logging notifications.

7. Perform one requested scenario at a time.
8. Export the notification log.

Acceptable formats

Best to worst:

- **Best**: raw notification byte dumps per characteristic in hex
- **Also good**: exported notification log from nRF Connect
- **Acceptable**: base64 payloads with clear labeling for EMG vs IMU characteristic
- **Fallback only**: screenshots or screen recordings of notification values if export is unavailable

For software fixture tests, raw bytes or exported logs are much better than screenshots.

Minimum metadata to include with every capture

For each capture file or message, include:

- scenario name
- date/time
- board used
- firmware file name: `esp32_ble_sleeve.ino`
- characteristic UUID the payload came from
- whether the payload was captured during normal operation or a forced fault condition

Important notes for the team

- The firmware sends **EMG notifications as 22-byte packets**.
- The firmware sends **angle/IMU notifications as 12-byte packets**.
- The angle value is the raw AS5600 12-bit angle or `0x7FFF` when the encoder read fails.
- The timestamp is the ESP32 `millis()` value at send time.
- The checksum is a simple XOR byte checksum.

What is most useful to software right now

If the team cannot produce every scenario, send these first:

1. idle stream
2. single contraction
3. encoder fault / no-magnet case

Those three alone are enough to lock the first real hardware fixtures into the parser and connector tests.

If nRF Connect is not available

If they cannot export BLE notifications directly, the next-best option is:

1. connect with the app using the BLE debug screen
2. record the Serial Monitor output at `115200`
3. record a phone screen video of the BLE debug page while the scenario runs
4. note exactly which scenario was being performed

That fallback is useful for diagnosis, but it is **not** as good as raw BLE payload export for fixture-based automated tests.